DATA INGESTION

FERRAMENTAS DE STREAMING - APLICAÇÃO FLUME

13

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

LISTA DE FIGURAS

Figura 1 - Generic real time systems. 8
Figura 2 - Arquitetura Streaming 10
Figura 3 - Ingestão batch ou stream 11
Figura 4 - Operadores Apache Flink 12
Figura 5 - Origens e destinos de dados para o Apache Flink 12
Figura 6 - Operação de paralelismo do Apache Flink 13
Figura 7 - Operação de paralelismo com estado do Apache Flink 15
Figura 8 - Exemplo de operação com estado do Apache Flink 16
Figura 9 - Arquitetura do Apache Flink 16
Figura 10: Exemplo de Tópico com 4 partições e dois produtores 21
Figura 11: Funcionamento Apache Flume. 23
Figura 12: Estrutura Apache Flume Event 23
Figura 13: Estrutura Apache Flume Agent 24
Figura 14: Exemplo de arquitetura usando Stream Analytics. 27
Figura 15: Painel do Stream Analytics 28
Figura 16: Criação de um Hub de Eventos do Stream Analytics. 28
Figura 17: Flow de eventos do Stream Analytics. 29
Figura 18: Arquitetura Alto Nível Kinesis Data Stream. 31
Figura 19: Checando versão do Java 35
Figura 20: Binários do Kafka 37
Figura 21 - Alteração do diretório de logs do Kafka 38
Figura 22 - Adiciona tópico que permite deletar tópicos 38
Figura 23 - Status do serviço do Kafka 40
Figura 24 - Criado o tópico no Kafka 41
Figura 25 - Producer e Consumer no Kafka em funcionamento 42

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

LISTA DE CÓDIGOS

Código-fonte 1 - Conexão SSH com servidor Linux. 35
Código-fonte - 2 Instalação do JDK 11 Fonte: Elaborado pelo autor 35
Código-fonte 3 - Atualizando os pacotes do Linux 36
Código-fonte 4 - Adicionando usuário Kafka 36
Código-fonte 5 - Atribuindo permissões de sudo ao usuário Kafka..... 36
Código-fonte 6 - Mudando conexão para usuário Kafka 36
Código-fonte 7 - Cria diretório Downloads 36
Código-fonte 8 - Download do Kafka..... 37
Código-fonte 9 - Cria diretório Kafka e acessa o diretório..... 37
Código-fonte 10 - Descompacta os binários do Kafka 37
Código-fonte 11 - Edita o arquivo server.properties..... 38
Código-fonte 12 - Arquivo de configuração do Zookeeper 39
Código-fonte 13 - Cria arquivo de serviço do Kafka..... 39
Código-fonte 14 - Conteúdo do arquivo de serviço do Kafka 40
Código-fonte 15 - Iniciando o serviço do Kafka..... 40
Código-fonte 16 - Checando status do serviço do Kafka 40
Código-fonte 17 - Habilitando o Zookeeper..... 40
Código-fonte 18 - Habilitando o Kafka..... 41
Código-fonte 19 - Cria um tópico no Kafka 41
Código-fonte 20 - Adiciona um evento com texto no producer do Kafka..... 41
Código-fonte 21 - Leitura do evento com texto pelo consumer do Kafka 42

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

SUMÁRIO

1 FERRAMENTAS DE STREAMING - APLICAÇÃO FLUME 5
1.1 APACHE FLINK 11
1.2 APACHE KAFKA 19
1.3 APACHE FLUME 22
1.4 AZURE STREAM ANALYTICS 25
1.5 KINESIS DATA STREAMS 30
2 HANDS-ON 35
Referências 43

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

1 FERRAMENTAS DE STREAMING - APLICAÇÃO FLUME

Os dados estão surgindo e fluindo de todos os lados em nosso dia a dia, por meio de smartphones, transações bancárias, compras on-line, modelos preditivos e afins. Em 2014, a EMC, em parceria com a IDC, lançou a sétima edição do estudo anual Digital Universe, que pode ser encontrado neste link https://www.dell.com/ptbr/dt/corporate/newsroom/announcements/2014/04/20140409-01.htm.

O estudo afirma que o universo digital está dobrando de tamanho a cada dois anos, passando de 44 bilhões de gigabytes em 2020.

Para facilitar a interpretação dos números e colocar isso em perspectiva, é: se um gigabyte fosse um galão de água, em 2020, levaria apenas 2 segundos para encher todos os espaços de uma casa média.

Isso mostra como o mundo está girando rápido e, embora nossa noção de Big Data exista há algum tempo, cada dia surgem novas tecnologias capazes de acelerar o consumo dos dados por tomadores de decisões, por algoritmos que devem ofertar algo para os clientes ou detectar atividades fraudulentas em uma transação bancária.

Dados temos aos montes, conseguimos saber muito sobre as pessoas, empresas, produtos e serviços sem nem os contratar, apenas cruzando inúmeras fontes de dados sobre isso; podemos gerar uma visão extremamente qualitativa que possa nos dizer se devemos comprar, o que devemos esperar e o que devemos fazer. Tudo isso com dados e, agora, de forma acelertada, pois como falei, o mundo está girando rápido, não saber o que o cliente quer, onde ele quer ser atendido, como ele quer ser tratado, que produtos vender para ele e qual preço cobrar pode decretar o fim de empresas, mas para fazer isso, não podemos executar um modelo mensalmente, semanalmente e, em alguns casos, nem mesmo diariamente, pois isso seria perder clientes para a concorrência, deixar de ganhar dinheiro, deixar seu cliente insatisfeito.

Muitos dos sistemas que precisamos trabalhar, como carros em movimento emitindo sinais de GPS, transações financeiras, intercâmbio de sinais de torres de telefonia celular e pessoas utilizando seus smartphones,

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

tráfego na web, registro de máquinas, medições de sensores industriais e gadgets de modo geral, funcionam com fluxo contínuo de eventos. Se conseguirmos a capacidade de analisar de forma eficiente esses dados de streaming em larga escala, estaremos em uma posição muito favorável para tomada de decisão e vender, comprar e agir de forma mais rápida com dados.

Precisamos acompanhar os eventos de nossos tempos. O momento agora requer cada dia mais agilidade na hora de usar dados, pois isso pode significar sair na frente do concorrente, salvar vidas, tornar a vida das pessoas melhor.

Parece simples, mas não é tanto assim. Precisamos de ferramentas apropriadas, arquitetura de dados que consiga suportar esse trânsito de dados imediato e entregar os dados prontos, processados a quem precisa deles naquele momento.

Nos próximos tópicos, entenderemos como tudo isso funciona e como fazer ambientes, ingestão e processamento de dados.

O QUE É UM SISTEMA EM REAL TIME

O que faz um sistema ser streaming, ser real time? Os sistemas em real time existem há décadas, mas se tornaram mais populares com o advento da internet. Mas com isso, vem a ambiguidade e o debate sobre o que é real time e o que não é.

Alguns exemplos dessa ambiguidade podem ser encontrados aos montes; dois segundos após o real time, ainda é real time? Quanto segundos ou minutos podem ser considerados real time? Para *Andrew Psaltis*, no livro *Streaming data*, isso pode ser traduzido na tabela a seguir:

Classificação	Exemplo	Latência medida em	Tolerância para atrasos
Hard	Marcapasso, freios de emergência, chamadas de emergências.	Microssegundos, milissegundos.	***Nenhuma***, risco de perda de vida, falha drástica em sistema.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Soft	Sistemas de reservas on-line, cotações de ações.	Milissegundos, segundos.	***Baixa***, sem risco de falha, sem riscos de vida.
Near	Automação residencial, chamadas de vídeo.	Segundos, minutos.	***Alta***, sem falha no sistema, sem risco de vida.

Quadro 1 - Classificação de ingestão de dados em streaming
Fonte: Streaming data, Cap. 1

Um sistema real time deve atender a um propósito/necessidade de negócios. E essa necessidade pode ser medida por meio do impacto que um atraso pode causar a esse negócio. Pense em uma central de atendimento onde os atendentes recebem as ligações de emergências policiais. Pessoas em apuros estão ligando pedindo ajuda; quanto tempo é aceitável esperar até que sua ocorrência seja transmitida a uma viatura próxima de você? Milissegundos, segundos, minutos?

E se fosse uma emergência médica? Quanto tempo é aceitável até que os dados da sua emergência cheguem até a ambulância para ela possa se deslocar até você?

Você trabalha no setor de tecnologia de uma indústria química onde há reatores, caldeiras, entre outros equipamentos, que podem se tornar perigosos caso ocorra alguma falha que pode levar a um incêndio ou explosão. Quanto tempo é aceitável até que alguém seja alertado que algo está errado?

Imagino que todos devem ter respondido que o tempo aceitável é o menor possível, de preferência que seja imediato. Ao cadastrar a ocorrência ou emergência, que seja disparado imediatamente para a viatura/ambulância.

Isso é real time.

É possível identificar sistemas hard real time com certa facilidade. Eles são popularmente encontrados em sistemas embarcados e possuem requisitos de tempo rígidos, que, caso não sejam atendidos, podem ocasionar em falha total de um sistema ou riscos à vida.

Agora, determinar se um sistema é soft ou near real time, apresenta um pouco mais de dificuldades na da classificação. Vamos trazer alguns exemplos:

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

- Você segue uma pessoa no X (antigo twitter) e, momentos depois, vê essa postagem no seu dispositivo móvel;
- Você está monitorando preços de passagens para Nova Iorque pelo seu site favorito;
- Ou está monitoramento o sobe e desce da bolsa de valores para acompanhar o andamento das suas ações.

Veja a imagem abaixo:

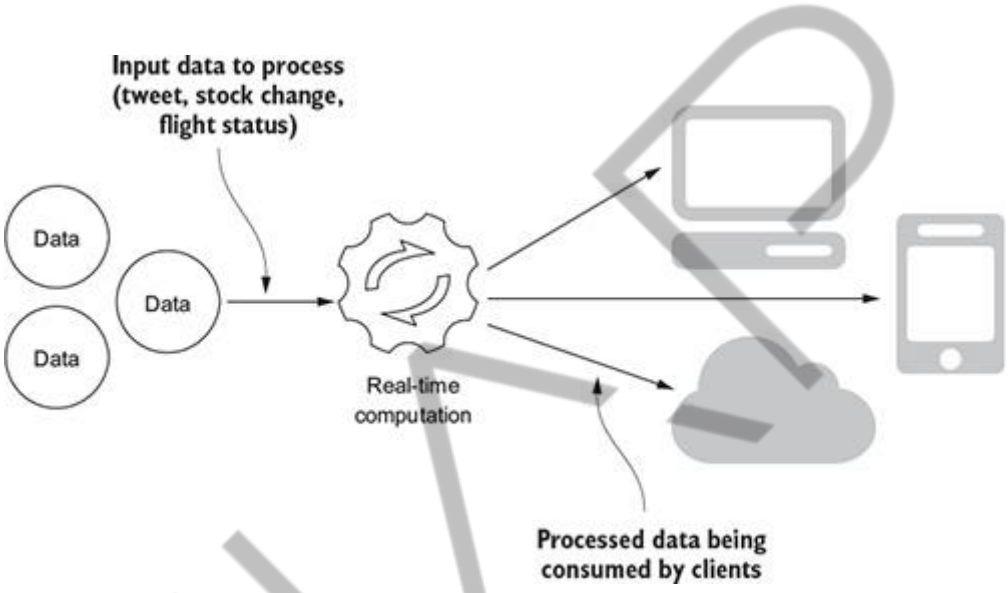


Figura 1 - Generic real time systems.
Fonte: Streaming data, Cap. 1.

Mas e se alguém postar um tweet e você não visualizar imediatamente? Ou aparecer uma oferta de voos e você não olhar no mesmo momento que foi postado? E se as ações que você tem em carteira oscilarem, mas você não consegue ver devido à lentidão do wi-fi ou porque estava em um voo? Para algumas pessoas, tudo bem não ver imediatamente, é tolerável alguns minutos para obter esse tipo de informação. Mas qual tipo seriam esses exemplos? Near real time ou soft real time?

Veja que existe uma linha tênue entre os dois tipos, enquanto nosso modelo hard real time é mais fácil de identificar, os outros dois exemplos são mais complexos, e o que vai determinar como são classificados é o consumidor desse dado com sua expectativa e sua percepção do atraso percebido.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Mas processar dados em real time e apresentar dados em real time podem ser coisas diferentes. Por exemplo, os dados do X (antigo Twitter) são real time, são postados imediatamente, os dados de anúncios de passagens aéreas são real time, pois a partir do momento que são publicados, estão disponíveis para consumo.

Observe que pensar em como o dado será disponibilizado é mais importante do que pensar em hard real time, soft real time ou near real time, porque precisamos que o dado esteja disponível para o consumidor no momento em que ele precisar.

Nesse caso, quando você abre seu aplicativo do X e atualiza o feed, você recebe os tweets mais recentes da sua rede de contatos, alguns com milissegundos, outros com segundos, outros com minutos e alguns com horas ou dias, mas são os tweets da sua rede disponibilizados quando você os solicitou, bem como as passagens aéreas, você pode entrar no site de compra de passagens agora e verá as promoções mais recentes, algumas mais recentes que outras, mas quando você precisar estará disponível.

ARQUITETURA REAL TIME

Um exemplo de arquitetura real time é apresentado por Andrew Psaltis no livro Streaming Data, onde são contemplados vários passos da aquisição dos dados até a apresentação/entrega ao cliente.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

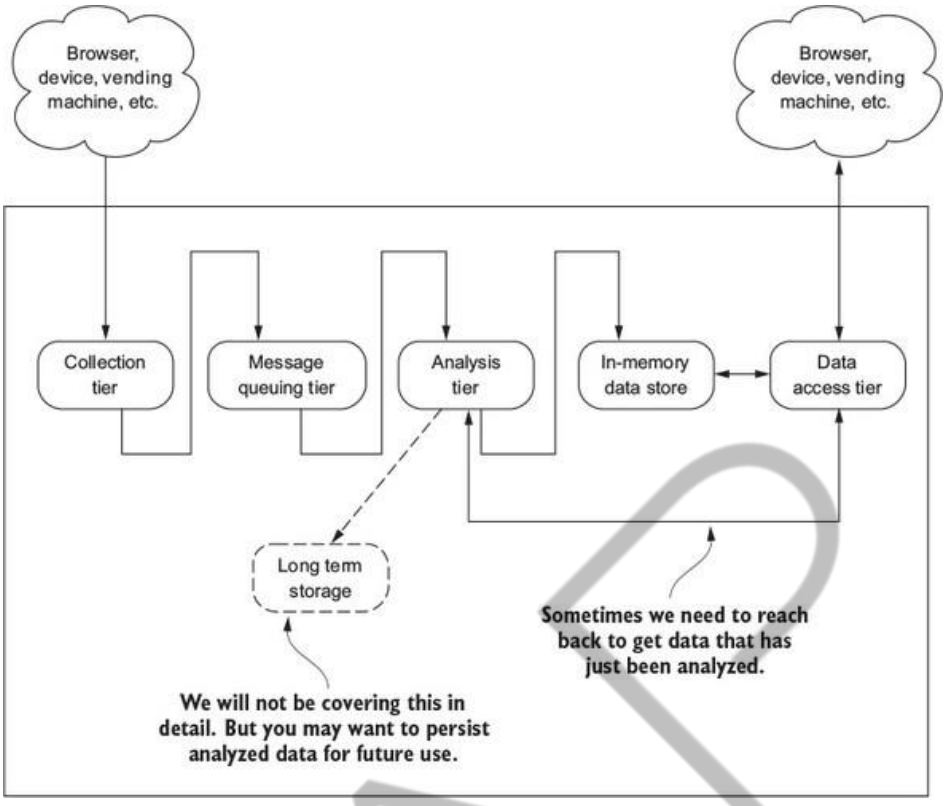


Figura 2 - Arquitetura Streaming
Fonte: Streaming data, cap. 1

Aqui nós temos nossos blocos de montar para compor uma arquitetura que resolva um problema específico.

FERRAMENTAS PARA STREAMING

A escolha de uma ferramenta para o projeto deve levar em consideração diversos fatores, como o que é sucesso para esse projeto, por exemplo: ter dados carregados em real time com tolerância mínima e escalabilidade sempre que precisar. Além disso, devemos considerar custos, se precisamos utilizar infraestrutura on-premisse por conta de alguma restrição da empresa ou se podemos utilizar cloud e alguma ferramenta paga pelo consumo, por exemplo.

Em termos técnicos, o propósito das ferramentas quase sempre é o mesmo, mas é algum detalhe técnico que torna uma ferramenta mais adequada ao seu projeto em relação às outras. Neste tópico, vamos explorar algumas das ferramentas mais populares e atuais do momento, a ideia é explanar seu funcionamento, arquitetura, pontos fortes e fracos de cada uma para passar um panorama geral da caixa de ferramentas da ingestão de dados em real time.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

1.1 APACHE FLINK

Quando pensamos em ingestão de dados, pensamos em fluxos. Um fluxo de dados trata eventos de dados de forma que percorram um fluxo de sua origem até seu destino. Podemos organizar nossos fluxos de dados de forma limitada ou ilimitada ou, em outras palavras, batch ou real time.

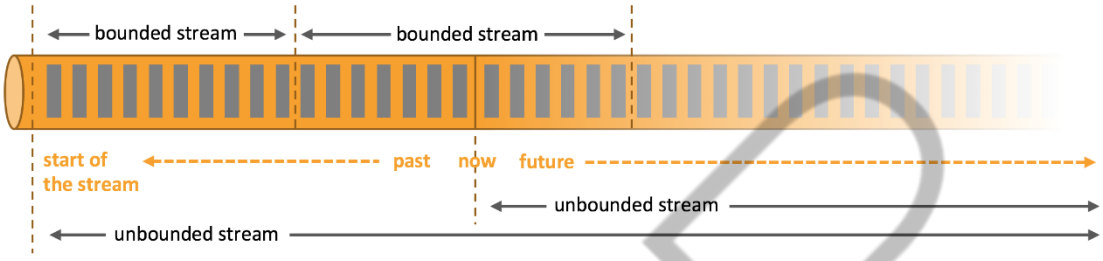


Figura 3 - Ingestão batch ou stream
Fonte: Documentação oficial Apache Flink

Uma dessas opções de fluxo é a ingestão de dados em batch, limitada, pois seus dados podem ser carregados em sua totalidade e depois carregado apenas o que há de novo, enquanto a outra opção será o foco desse material, a ingestão em streaming ou ilimitada, pois o fluxo não acaba, o fluxo processa os dados continuamente à medida que eles chegam.

No Apache Flink, as aplicações são compostas por fluxos de dados de streaming que forma grafos direcionados com início em uma ou mais fontes de dados e terminam também em um ou mais destinos chamados de Sink.

Observe o código e o fluxo apresentados abaixo:

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

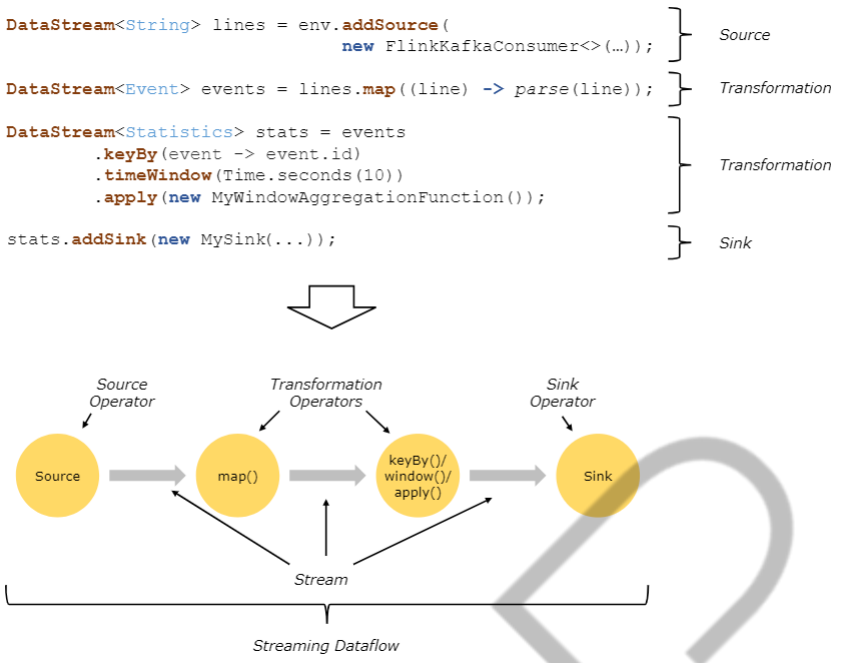


Figura 4 - Operadores Apache Flink
Fonte: Documentação oficial Apache Flink

Como observamos no código e fluxo apresentados acima, temos uma origem de dados (*source operator*), temos operadores de transformações como map () e demais transformações como definição de id, definições de tempo em segundos e funções de agregação.

O Apache Flink, pode consumir dados de fontes streaming, como filas de mensagens, sinais de GPS, logs distribuídos, mas também pode consumir de dados históricos de uma variedade de fontes de dados, e da mesma forma, pode enviar dados para uma ampla variedade de sistemas.

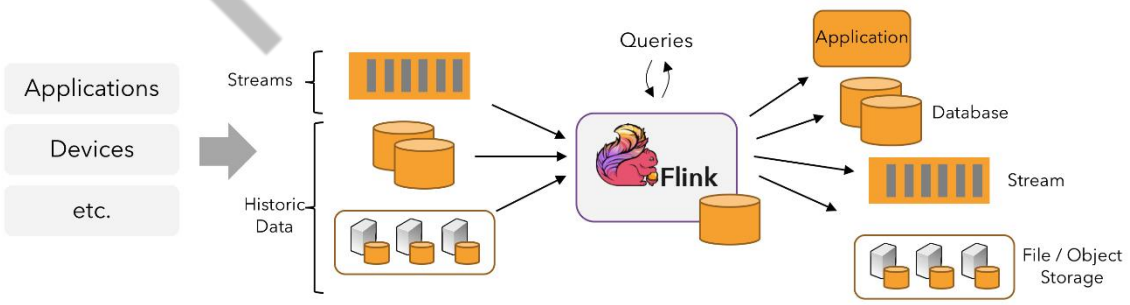


Figura 5 - Origens e destinos de dados para o Apache Flink
Fonte: Documentação oficial Apache Flink

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Fluxos de dados paralelos

O Apache Flink opera de forma paralela e distribuída. Isso quer dizer que durante a execução de um programa Apache Flink, um fluxo possui uma ou mais partições de fluxo e cada operador possui uma ou mais subtasks de operador.

As subtasks do operador são independentes umas das outras e são executadas em threads de CPU diferentes e muitas vezes em máquinas ou containers diferentes. O número de subtasks do operador é o número de paralelismo desse operador. Cada operador, mesmo sendo do mesmo programa, pode conter diferentes níveis de paralelismo. Abaixo, podemos ver um exemplo do paralelismo do Apache Flink.

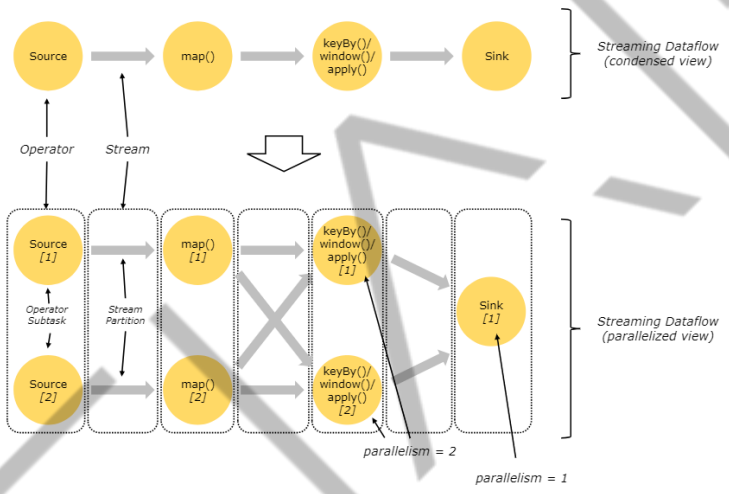


Figura 6 - Operação de paralelismo do Apache Flink
Fonte: Documentação oficial Apache Flink

Os fluxos podem transferir dados entre operadores de forma um para um, também chamado de encaminhamento ou redistribuição.

Ambos os fluxos podem ser observados na figura 6, entre os operadores Source e map (), o particionamento, bem como a ordem dos elementos, são preservados. Isso significa que a subtasks 1 do operador map () verá os mesmos elementos na mesma ordem em que foram produzidos pela subtasks 1 do operador source.

Na redistribuição de fluxos, como ocorre entre os operadores map () e keyby/window e também entre keyby/window e sink da figura 6, o particionamento dos fluxos é alterado. Ou seja, cada subtasks do operador envia

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

dados para diferentes subtasks de destino, a depender da transformação selecionada.

Processamento de dados oportuno

Uma das grandes sacadas na ingestão de dados de modo geral é poder processar dados históricos e real time em um único código, atendendo aos requisitos do projeto sem precisar alterar código ou ter mais de um processo de ingestão rodando para a mesma origem de dados.

Um dos pontos importantes aqui é se atentar à ordem em que os eventos ocorreram e não exatamente à ordem que foram entregues para processamento, pois os dados devem estar completos ou prontos para serem processados.

Imagine uma ingestão real time conectada à uma tabela de vendas, mas que por uma regra de negócio, a venda só é concretizada após emissão da nota fiscal. Em tese, sua carga de dados só pode ser executada após a emissão da nota fiscal, pois ainda falta uma etapa para concluir essa venda. Casos como esse podem ser resolvidos com um processamento que leva em consideração um registro de data e hora, considerando o tempo que a nota leva para ser emitida e, em caso de falha, a venda ser rejeitada. Por exemplo, o processo de venda submete os dados para a receita para emissão da nota imediatamente, se não houver um retorno dentro de 60 segundos, a venda fica com status de aguardando, nosso fluxo pode considerar o tempo e o status para levar somente os dados prontos, note que isso leva também dados históricos.

Processamento de fluxo com estado

O Apache Flink pode operar com o estado. Isso significa que a forma como um evento é tratado pode depender do efeito acumulado dos eventos anteriores a ele. Esses eventos podem ter propósito simples, como contar eventos por minuto para visualização em um dashboard ou até mesmo conter um modelo estatístico que possa classificar um evento como possivelmente fraudulento.

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Como dito anteriormente, uma aplicação Apache Flink é executada de forma paralela em um cluster distribuído. As instâncias paralelas de um operador serão executadas de forma independente, em threads separadas e, em geral, serão executadas em máquinas ou containers diferentes.

O conjunto de instâncias paralelas de um operador com estado trata-se de um armazenamento de valores chave fragmentados. Cada instância paralela é responsável por manipular eventos para um grupo específico de chaves, e o estado dessas chaves é mantido localmente.

O diagrama a seguir demonstra o trabalho em execução com paralelismo de dois nos três primeiros operadores, terminando em um coletor que tem paralelismo de um. A terceira operadora tem estado e você pode ver que está ocorrendo uma confusão de rede totalmente conectada entre a segunda e a terceira operadoras. Isso está sendo feito para particionar o fluxo por alguma chave, de modo que todos os eventos que precisam ser processados juntos o sejam.

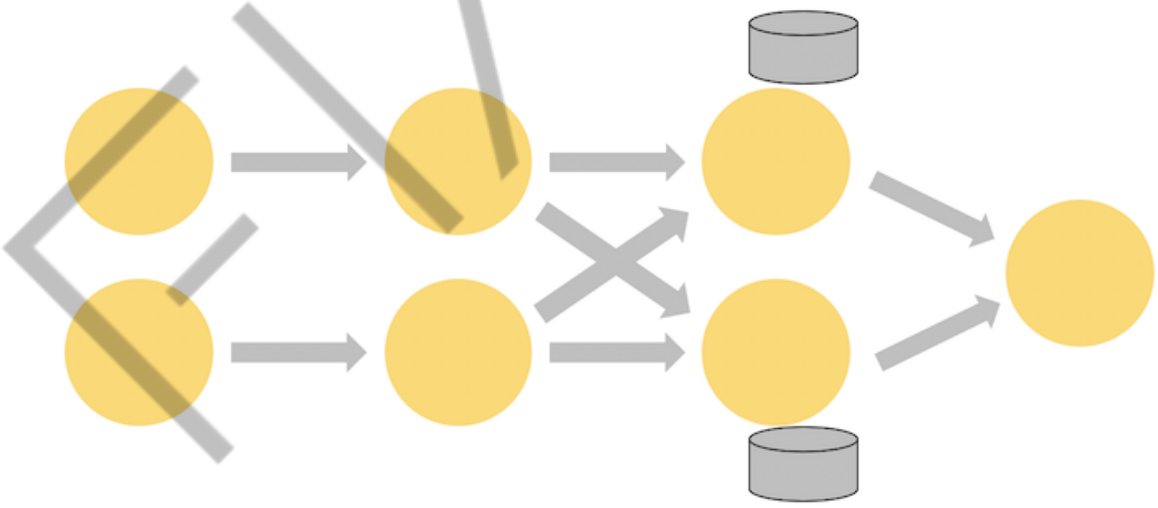


Figura 7 - Operação de paralelismo com estado do Apache Flink
Fonte: Documentação oficial Apache Flink

O estado é sempre acessado localmente, o que ajuda os aplicativos Flink a obter alto rendimento e baixa latência. Você pode optar por manter o estado no heap da JVM ou, se for muito grande, em estruturas de dados em disco.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

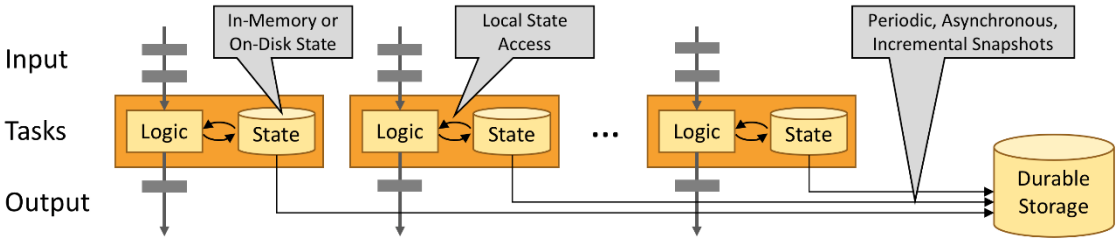


Figura 8 - Exemplo de operação com estado do Apache Flink
Fonte: Documentação oficial Apache Flink

Arquitetura Apache Flink

Apache Flink é um sistema distribuído que se integra facilmente com todos os gerenciadores de clusters mais populares, como Hadoop Yarn e Kubernetes, mas pode ser configurado para ser um cluster independente ou até mesmo como uma biblioteca.

Vamos apreciar a arquitetura do Apache Flink na imagem abaixo:

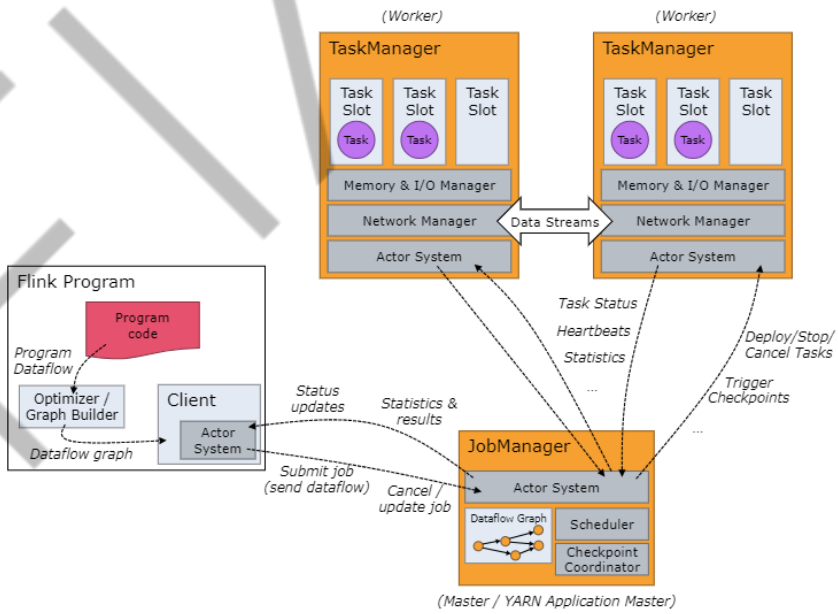


Figura 9 - Arquitetura do Apache Flink
Fonte: Documentação oficial Apache Flink

No Apache Flink, o Client submete um fluxo de dados ao JobManager e após isso, pode ser executado de duas formas, modo conectado ou modo

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

desanexado. No modo desanexado, ele submete o fluxo de dados e fica off-line após isso, no modo conectado, ele recebe relatórios de progresso da execução do fluxo.

O JobManager e os TaskManagers, de diferentes formas: diretamente em máquinas, como um cluster independente, em containers ou gerenciados por estruturas de recursos como o Apache Yarn. TaskManagers se conectam aos JobManagers, enviando um sinal como disponível e assim passam a receber tasks.

JobManager

Agora vamos conhecer melhor os componentes essenciais do funcionamento do Apache Flink. O primeiro deles, o JobManager, possui diversas responsabilidades, entre elas, a coordenação da execução distribuída de aplicativos Apache Flink. Ele quem decide quando agendar a próxima task, ou conjunto de tasks, reage a tasks concluídas com sucesso ou falha de execução, coordena checkpoints e recuperação em caso de falhas. Para fazer tudo isso, ele conta com três recursos diferentes:

- ResourceManager – Responsável pela alocação, deslocação e provisionamento de recursos em um cluster Apache Flink. Ele gerencia slots de tasks, que são basicamente unidades de agendamentos de recursos em um cluster Apache Flink. O Apache Flink pode implementar vários ResourcesManagers para diferentes ambientes de execução e provedores de recursos como Apache Yarn, Kubernetes e implantações autônomas;
- Dispatcher – Responsável por oferecer uma interface REST para envio de aplicativos Apache Flink para execução, além de iniciar um novo JobMaster para cada carga de trabalho enviada. Dispatcher também executa o Flink WebUI, interface que oferece informações sobre execuções de cargas de trabalho;
- JobMaster – JobMaster é responsável por gerenciar a execução de um único JobGraph. Várias cargas de trabalho podem executar

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

simultaneamente em um cluster Apache Flink, cada um com seu JobMaster.

Sempre haverá ao menos um JobManager. Uma configuração de alta disponibilidade pode ter vários JobManagers, dos quais um é o líder em execução e os demais em estado de espera para assumir, caso haja falha no JobMaster líder.

TaskManagers

Os TaskManagers são os executores do fluxo de dados, armazenam em buffer de memória e trocam os fluxos de dados. Sempre deve haver ao menos um TaskManager. O nível mais granular de um agendamento de recursos em um TaskManager é um slot de task. O número de slots de tasks em um TaskManager indica a quantidade de tasks de processamento simultâneo contidas nesse TaskManager.

Conectores do Apache Flink

O Apache Flink fornece uma série de conectores/sinks para as mais diversas fontes de dados, veja a seguir:

Apache Kafka	Apache Cassandra	Amazon DynamoDB	Amazon Kinesis Data Streams	Amazon Kinesis Data Firehouse
Data Gen	Elasticsearch	OpenSearch	FileSystem	RabbitMQ
Google PubSub	Hybrid Source	Apache Pulsar	JDBC	MongoDB

Quadro 2 - Conectores disponíveis no Apache Flink
Fonte: Apache Flink, documentação oficial

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

1.2 APACHE KAFKA

Uma boa definição de Streaming de eventos vem da documentação oficial do Kafka. “Streaming é o equivalente digital do sistema nervoso central do corpo humano.” E nos oferece também um ótimo ponto de vista sobre esse tema, onde as empresas estão cada vez mais definidas e automatizadas por software, onde muitas vezes o utilizador de software é mais software.

Isso nos mostra que muitas vezes nossos dados estão sendo consumidos não por humanos, mas por máquinas, por softwares que vão fazer algo com aquele dado que foi gerado em um sistema de origem e foi transportado pelo nosso processo de streaming. Esse processo de streaming garante que o dado esteja no lugar certo no momento certo.

E uma das soluções que temos atualmente para atender a esse tipo de demanda é o Apache Kafka. Trata-se de uma plataforma de streaming de eventos que combina três recursos principais para implementação em projetos de streaming de dados de ponta a ponta em uma única solução. Veja abaixo os três recursos principais do Kafka:

- Publicar (Ler) e assinar (escrever) fluxos de eventos, o que inclui importação e exportação de contínua de dados de outros sistemas;
- Armazenar fluxos de eventos de forma durável e confiável pelo tempo que for necessário;
- Processar fluxos de eventos à medida que ocorrem (real time) ou retrospectivamente (histórico).

E essas funcionalidades são fornecidas de forma distribuída, altamente escalável, elástica, tolerante a falhas e de forma segura. O Apache Kafka pode ser instalado em hardware físico, virtualizado e containers, tanto em infraestrutura local quanto em provedores de nuvem, e ainda pode ser contratado como serviço em alguns fornecedores de cloud.

A estrutura básica do Apache Kafka é seu sistema distribuído e sua composição de servidores e clientes que se comunicam por meio de protocolo TCP de alto desempenho.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Servidores – O Kafka é executado como um cluster de um ou mais servidores que podem estar distribuídos em datacenters e/ou regiões de nuvem. Alguns desses servidores formam uma camada de armazenamento chamada de corretores. Já os outros servidores executam o Kafka Connect, responsável por integrar o Kafka aos seus sistemas existentes, como bancos de dados relacionais e NoSQL ou até mesmo outros clusters Kafka. O Kafka pode ser implantado em ambientes de missão crítica, onde usando sua tolerância a falhas e escalabilidade, podemos configurar os servidores para atuarem em caso de falha de outro servidor, mantendo as operações contínuas sem qualquer perda de dados.

Clientes – Permitem que seja desenvolvido aplicações e microsserviços distribuídos que façam leitura, escrita e processamento de fluxos de eventos em paralelo, escala e de maneira tolerante a falhas, mesmo em caso de problemas na máquina ou de rede. O Kafka já traz alguns clientes incluídos por padrão e muitos mais são fornecidos pela comunidade Kafka. Os clientes estão disponíveis em Java e Scala e há também a biblioteca Kafka Streams de alto nível para linguagens Go, Python, C/C++, entre outras linguagens, e REST APIs.

Principais conceitos e terminologia

Um evento registra algo que aconteceu na empresa, uma venda, um cancelamento, uma postagem, uma leitura de dados de sensores aeroespaciais, o que também pode ser chamado de registro.

Ao ler ou gravar dados no Kafka, fazemos isso na forma de eventos. Conceitualmente, um evento no Kafka possui uma chave, um valor, um carimbo de data/hora e cabeçalhos de metadados opcionais.

Na arquitetura do Kafka, temos produtores e consumidores. Produtores são aplicações clientes que fazem publicação (operações de escrita) no Kafka, enquanto consumidores são aquelas aplicações que assinam (leem e processam dados) esses eventos. No Kafka, produtores e consumidores são totalmente desassociados, o que é um elemento chave no design para alcançar a alta escalabilidade pela qual o Kafka é bastante conhecido. Um exemplo dessa escalabilidade são os produtores nunca precisarem esperar pelos consumidores.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Os eventos têm um nível de organização e armazenamento em tópicos, que fornece durabilidade e acesso aos dados de forma configurável a nível de tópico. Um tópico, de forma simplificada, podemos fazer alusão a um diretório com arquivos, onde o diretório seria o tópico e os arquivos seriam os eventos dentro dele. Por exemplo, um tópico chamado Pagamentos; dentro dele, teríamos os eventos de pagamentos. Os tópicos do Kafka são sempre multiprodutores e multiassinantes, ou seja, um tópico pode ter nenhum, um ou muitos produtores e assinantes.

Os eventos podem ser lidos quantas vezes forem necessárias, esses eventos não são excluídos após o consumo, como ocorre em outras ferramentas. Em vez disso, no Kafka, podemos configurar por quanto tempo o evento ficará retido antes de ser descartado, e essa configuração é feita por tópico.

Mas armazenar esses eventos por um período de tempo não é um problema, já que o Kafka armazena os tópicos distribuídos em buckets localizados em diferentes corretores Kafka. Quando um novo evento é publicado em um tópico, é anexado a uma das partições do tópico que faz parte.

Abaixo, podemos apreciar um exemplo de um tópico com quatro partições, P1, P2, P3 e P4. E temos dois clientes produtores publicando de forma independente novos eventos para o tópico. Eventos com a mesma chave, indicados pela cor da figura, são gravados na mesma partição. Mas observe que os mesmos produtores também podem gravar na mesma partição, se for o caso.

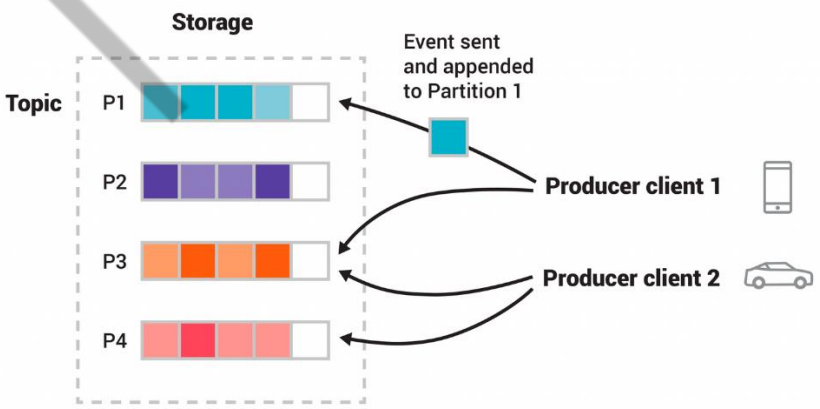


Figura 10: Exemplo de Tópico com 4 partições e dois produtores
Fonte: Documentação oficial Apache Kafka

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

APIs Kafka

O Kafka possui, além das ferramentas de linha de comando, utilizadas em tarefas de administração e gerenciamento da ferramenta, cinco APIs principais em Java e Scala, conforme apresentado abaixo:

- API Admin – API para gerenciar e inspecionar tópicos, corretores e outros objetos Kafka;
- API Consumer – API para assinar (ler) um ou mais tópicos e processar o fluxo de eventos produzidos para eles;
- API Kafka Streams – API para implementar aplicações e microsserviços de processamento de stream. Essa API fornece funções de alto nível para processar fluxos de eventos, incluindo transformações, operações com estado de agregações, joins, entre outras funções. A entrada pode ser lida de um ou mais tópicos e a saída também para um ou mais tópicos;
- API Kafka Connect – API para criar a executar conectores de importação e exportação de dados reutilizáveis que consomem (leem) ou produzem (gravam) fluxos de eventos de e para sistemas e aplicações externas e se integrar ao Kafka. Por exemplo, um conector para banco de dados Oracle pode capturar todas as alterações em um conjunto de tabelas. Na prática, dificilmente será necessário criar seu próprio conector, uma vez que na comunidade Kafka existem centenas de conectores prontos para uso.

Mais um produto Open Source, da comunidade técnica para quem desejar usar uma das mais conhecidas e usadas ferramentas de ingestão em real time do momento, traz muitos pontos positivos, tem uma arquitetura muito propícia à escalabilidade, o que faz desse produto um dos mais parrudos e interessantes que temos atualmente.

1.3 APACHE FLUME

O Apache Flume é um serviço que permite coletar, agregar, processar e mover grandes volumes de dados em ambiente distribuído e inserir esses dados

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

de eventos no HDFS (Hadoop Distributed File System) de forma simples e automatizada.

O Apache Flume pode ser utilizado em diversos casos de uso, como transportar dados de Mídia sociais, E-mails, Logs, dados adquiridos na web, como crawlers, e qualquer outra fonte de dados disponível.

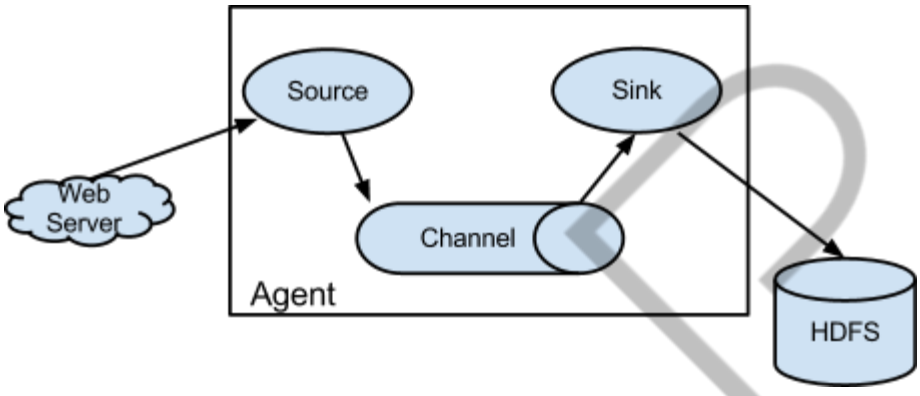


Figura 11: Funcionamento Apache Flume.
Fonte: Documentação oficial Apache Flume.

O modelo de fluxo de dados do Apache Flume é composto por:

- Flume Event – Um evento é a unidade básica de dados a serem transportados pelo Flume. É composto por uma matriz de bytes que deve ser transportada entre a origem e destino e pode ser enviada junto com cabeçalhos. Um típico evento no Flume teria a seguinte estrutura:

Figura 12: Estrutura Apache Flume Event.
Fonte: Documentação oficial Apache Flume.

- Flume Agent – Um agente é um processo daemon independente que roda na JVM (Java virtual machine) no Flume. Ele recebe dados (eventos) de clientes ou outros agentes e os encaminha para seu próximo destino (sink

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

ou agente). O Flume pode ter um ou mais agentes. Veja a figura abaixo que exemplifica esse conceito:

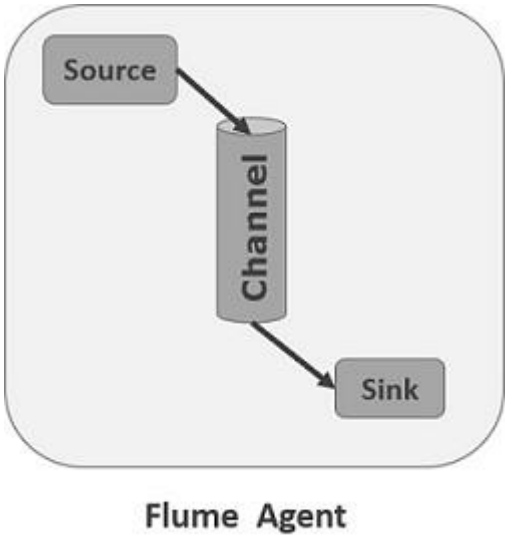


Figura 13: Estrutura Apache Flume Agent
Fonte: Documentação oficial Apache Flume

Um agente Flume é composto por três componentes principais, Source, Channel e Sink, que serão detalhados abaixo.

- Source – Componente de um Agente que recebe dados de geradores de dados e os transfere para um ou mais canais na forma de eventos Flume. O Flume tem suporte a vários tipos de fontes de dados e cada um recebe eventos de um gerador específico;
- Channel – Um canal é um armazenamento transitório que recebe os eventos da origem e os armazena em buffer até que sejam consumidos pelos coletores. Sua atuação é basicamente como uma ponte que liga as origens de dados (source) e os Sinks. Esses canais são totalmente transacionais e podem funcionar com qualquer quantidade de sources e Sinks;
- Sink – Um coletor armazena os dados em armazenamentos centralizados, como HBase e HDFS. Ele consome os dados (eventos) dos canais e os entrega ao destino. O destino do coletor pode ser outro agente ou o armazenamento central.

Um Agente pode ter um ou múltiplos Sources, Sinks e Channels.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Componentes Adicionais

Os componentes citados acima são componentes padrão do agente do Flume. Além desses componentes, temos outros que também possuem relevância na arquitetura do Flume, que vale a pena serem citados aqui:

- Interceptadores – São componentes usados para inspecionar ou até alterar eventos que estão sendo transferidos entre origem e um canal. Esses elementos são interessantes, pois podem ser usados como pontos que tragam alguma validação dos dados que podem servir como data quality, e até mesmo transformações mais simples, como alteração de data types e correções simples;
- Seletores de canais – São usados para determinar qual canal deve ser escolhido para transferir os dados no caso de haver múltiplos canais. Existem dois tipos de seletores de canais:
    - *Seletor de canal padrão* – Conhecidos como seletores de canal replicante, esse seletor replica todos os eventos em cada canal;
    - *Seletor de canal multiplexing* – Decide o canal para enviar um evento com base no endereço no cabeçalho desse evento.
- Processador de Sink – Usados para invocar um coletor específico do grupo selecionado de coletores. São usados para criar caminhos de failover para seus coletores ou eventos de balanceamento de carga em vários coletores de um canal.

Mais uma ferramenta incrível, com potencial para atender a projetos de baixa complexidade e até mesmo os mais complexos, de forma eficiente e com bastante dinamismo. Um dos principais benefícios de ferramentas open source é a autonomia que temos para trabalhar com elas. E com Flume não é diferente.

1.4 AZURE STREAM ANALYTICS

Vimos algumas ferramentas bem interessantes e open source até o momento, chegou a hora de dar uma olhada em uma ferramenta paga e auto gerenciada em cloud. Vamos conhecer melhor o Azure Stream Analytics, da

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Microsoft, que se trata de um mecanismo de processamento de fluxo de eventos projetado para analisar e processar grandes volumes de dados de streaming com latências superbaixas, na casa dos milissegundos. Além da ingestão, na camada de processamento podemos identificar relações e padrões nesses dados que podem vir de uma enorme variedade de origens como aplicativos, dispositivos móveis, sensores, sequência de cliques e feed de mídias sociais.

Os padrões detectados podem ser usados como gatilhos para disparo de ações e/ou iniciar fluxos de trabalho, criar alertas, envio de e-mails, armazenamento desse fluxo em condições especificas e outros fins.

Veja alguns exemplos de uso para o Azure Stream Analytics:

- Pipeline de dados streaming armazenando no Azure em formato Parquet;
- Aplicação controlada por eventos com banco de dados SQL e Azure Cosmos DB;
- Análise de fluxo e logs de telemetria em real time de aplicativos e dispositivos IOT (*Internet of things*);
- Painel em real time com Stream Analytics conectado ao Power BI;
- Detecção de anomalias para detectar picos, quedas e comportamentos anômalos em sensores;
- Análise geoespacial para gerenciamento de frotas e veículos sem motorista;
- Análise de sequência de cliques para determinar o comportamento de clientes em sites, aplicativos e ferramentas.

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

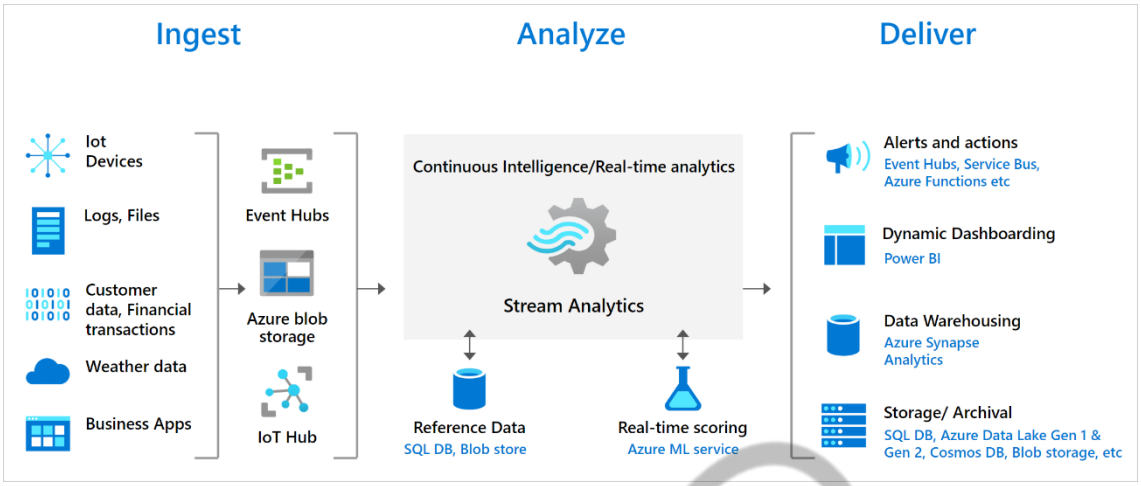


Figura 14: Exemplo de arquitetura usando Stream Analytics.
Fonte: Documentação oficial Azure Stream Analytics.

Com certeza um dos grandes benefícios do stream Analytics é a facilidade de criar pipelines de dados em Streaming com poucos cliques, se conectando a diversas origens de dados e roteando a saída para vários sistemas de armazenamento distintos ou até mesmo apresentando direto em um dashboard como Power BI.

Agora, vamos conhecer um pouco mais a forma de criar ingestão de dados usando Azure Stream Analytics. Um flow no Stream Analytics é formado por três componentes principais: entradas de dados, transformações e saídas. Mas isso não se limita a uma única entrada, única transformação e única saída, podemos ter várias entradas que podem ser unificadas, transformações diversas e, por fim, distribuir essa saída em ferramentas diferentes.

Ao se conectar no portal do Azure Stream Analytics, criaremos um job do stream Analytics (painel à esquerda da figura 14) e selecionamos No Code Editor. Veja que temos a opção de criar também via Query editor ingest data, analyze data e output data.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

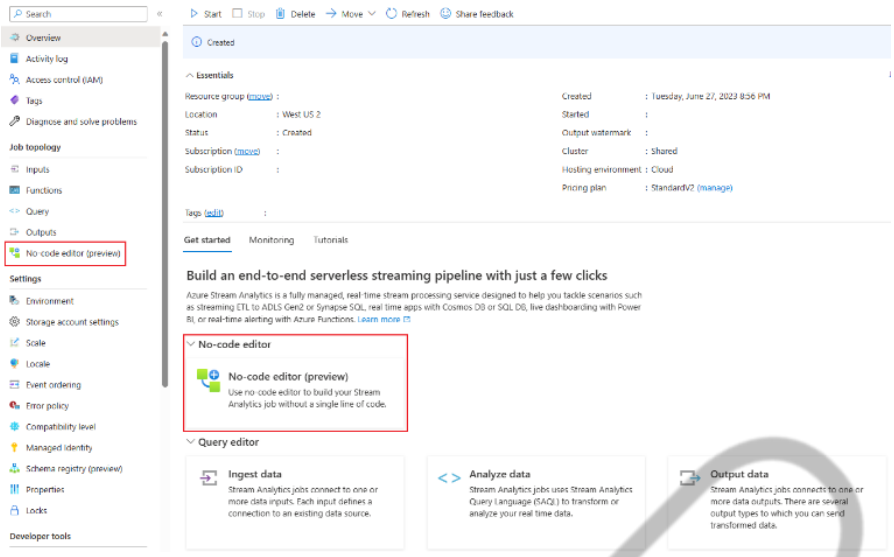


Figura 15: Painel do Stream Analytics
Fonte: Documentação oficial Azure Stream Analytics

No portal Hubs de eventos do Azure, abriremos uma instancia de Hubs de eventos e selecionaremos Processar dados, selecionamos também a primeira opção *Filter and Ingest to Synapse SQL*.

Fazemos a conexão entre os dados de origem (entrada), aplicamos os filtros (transformação) e a saída.

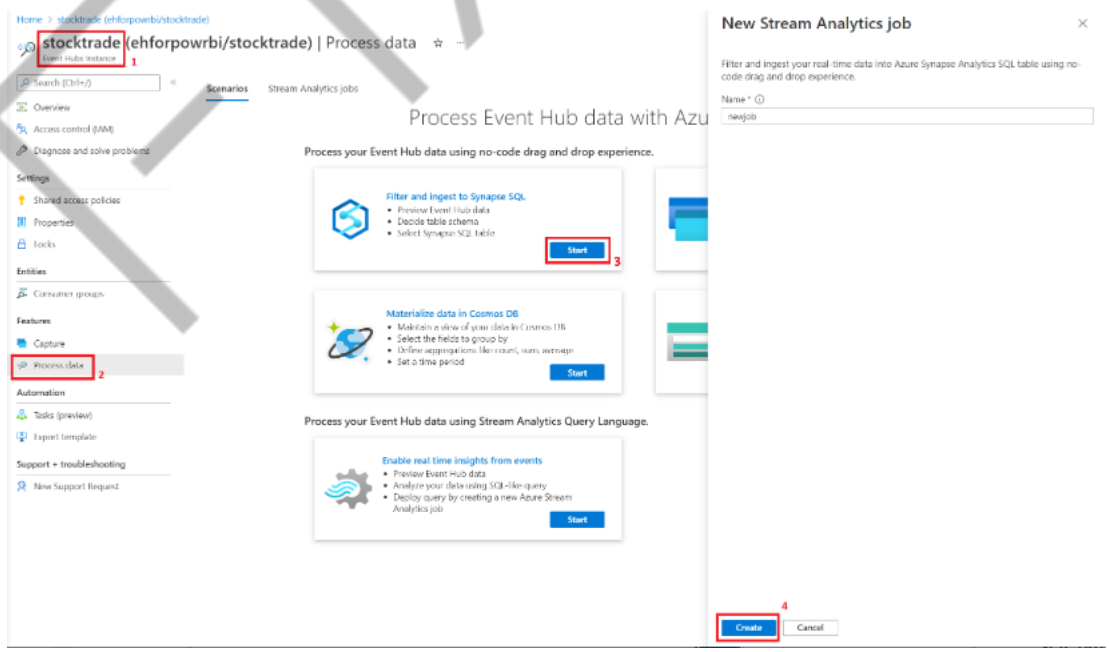


Figura 16: Criação de um Hub de Eventos do Stream Analytics.
Fonte: Documentação oficial Azure Stream Analytics.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Nesse exemplo, os dados de entrada são de valor de fechamento de ações da bolsa de valores e os dados são capturados da internet. Filtramos aquilo que venha do Google, ajustamos data types e passamos para o Azure Synapse.

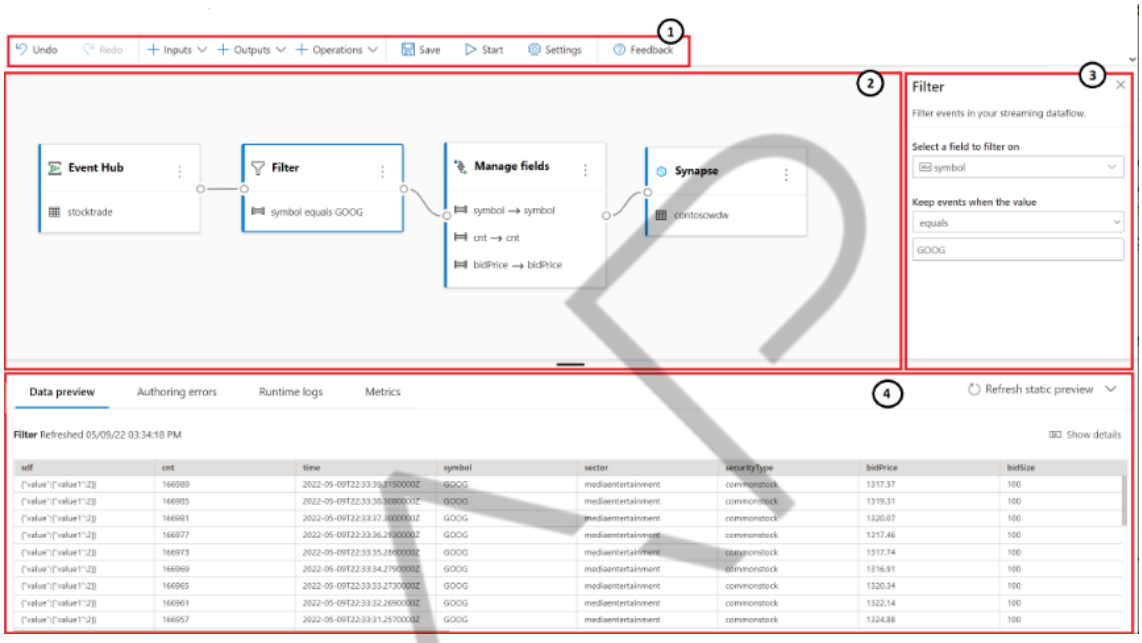


Figura 17: Flow de eventos do Stream Analytics.
Fonte: Documentação oficial Azure Stream Analytics.

Para usar o recurso de Editor sem código, temos algumas limitações na origem de dados. Temos suporte a três tipos de recursos:

Event Hub;

IoT Hub;

ADLS Gen2 (Azure Data Lake Storage Gen2)

Esses conectores disponíveis podem ter seus dados unidos à entrada de dados de referência. Um recurso que permite a conexão de dados estáticos ou que sofrem mudanças lentamente para enriquecer os dados stream capturados pelo hub de eventos. Esses dados podem vir de fontes como Azure Data Lake Storage Gen 2 e Bancos de dados SQL do Azure.

Mais uma ferramenta incrível para ingestão de dados em real time, fugindo um pouco do que vimos até agora, uma ferramenta que não é open source, mas

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

muitas vezes vale a pena investir em uma tecnologia que vai trazer mais agilidade e mais robustez às nossas entregas.

1.5 KINESIS DATA STREAMS

Assim como as demais ferramentas, o AWS Kinesis Data Streams é uma ferramenta para criar aplicações que coletam e processam grandes fluxos de dados em real time que podem ser executadas em instâncias do AWS EC2.

Esses dados coletados e processados podem ser armazenados em bancos de dados, dashboard, gerar alertas, alterar dinamicamente estratégias de preço e publicidade ou enviar esses dados para modelos de machine learning e outros serviços da AWS ou outros provedores de cloud.

O Kinesis Data Streams faz parte de uma plataforma de dados de Streaming de Kinesis da AWS que contém o Kinesis Data Firehose, Kinesis Vídeo Streams e o serviço gerenciado do Apache Flink, todos disponíveis na AWS.

Caso precise de maior poder computacional, podemos usar clusters AWS EMR para ler e processar os flow do Kinesis, usando ferramentas do ecossistema Hadoop, como: Pig, Hive, MapReduce, Hadoop Streaming API e Cascading. Podemos coletar dados do AWS S3, Amazon DynamoDB e HDFS em um cluster em execução.

A integração entre Kinesis Data Stream e EMR possui muitos benefícios quando pensamos em processamento de um grande volume de dados e em agilidade nesse processamento. Estaremos unindo uma ferramenta de streaming poderosa a um grande poder computacional que pode ser usado para tarefas que necessitam ser executadas com rapidez e segurança.

Arquitetura Kinesis Data Stream

Na figura 18, podemos ver a arquitetura onde temos os producers, que nada mais são do que as origens de entrada de dados, temos o Kinesis Data

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Stream e os consumers, que são os consumidores dos flow de dados em stream gerados.

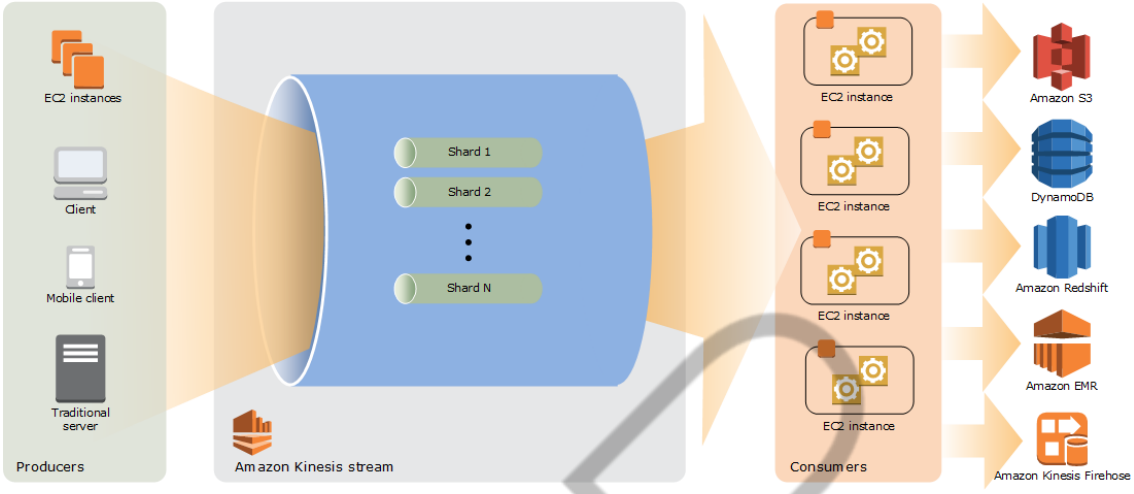


Figura 18 - Arquitetura Alto Nível Kinesis Data Stream.
Fonte: Documentação oficial Kinesis Data Streams

Fluxos de dados Kinesis

Um flow de dados Kinesis é composto por um conjunto de shards. Cada shard tem uma sequência de registros de dados e cada registro de dados possui um número de sequência atribuído pelos flows de dados do Kinesis.

Registro de dados

Um registro é a unidade de dados armazenado em um flow de dados Kinesis. Os registros de dados são compostos por um número de sequência, uma chave de partição e um blob de dados, que nada mais é que uma sequência imutável de bytes que pode ser de até 1 MB. O flow de dados não é responsável por inspecionar, interpretar ou alterar os dados no blob.

Produtores

São responsáveis por enviar registros de dados nos flows de dados do Kinesis. Por exemplo, um servidor Web enviando dados de log para um flow é um produtor.

Consumidor

Os consumidores obtêm registros do Kinesis e os processam. Esses consumidores são conhecidos como aplicações AWS Kinesis Data Streams.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Capacidade

Podemos definir uma capacidade que será provisionada e executada nos flows de dados ou podemos deixar configurado sob demanda, dessa forma, conforme os flow precisem de mais poder computacional, isso será escalado conforme a necessidade.

Período de retenção

Podemos definir um período em que os registros de dados ficarão acessíveis após serem adicionados ao flow. O período de retenção de um flow é de 24 horas após a sua criação, mas esse período pode ser aumentado para até 8760 horas (365 dias) ou reduzido conforme a necessidade.

Aplicação Kinesis Data Streams

Uma aplicação que consome um flow de dados e geralmente executada em uma instancia AWS EC2. Esses consumidores podem ser de dois tipos: consumidor de distribuição compartilhada e consumidor de distribuição aprimorada.

A diferença entre os dois tipos é basicamente a cota da taxa de transferência usada em distribuições aprimoradas que são de 2MB/s por leitura dedicada a uma aplicação com distribuição aprimorada. A aplicação com distribuição compartilhada compartilha essa mesma taxa com outras aplicações.

Uma saída do Kinesis pode ser a entrada de outro flow, o que nos permite criar arquiteturas complexas conectadas que processam dados de ponta a ponta em tempo real.

Shard

Um shard é uma sequência exclusivamente de registros de dados em um flow. Um flow é composto por um ou mais shards e cada um fornecendo uma unidade fixa de capacidade. Cada shard suporta até 5 transações de leituras por segundo com uma taxa total máxima de leituras de até 2MB por segundo e até 1000 registros por segundo para operações de escrita, com uma taxa total de até 1 MB por segundo, incluindo chaves de partição. A capacidade de dados de um flow é quantidade de shards especificados para o stream e a capacidade total do stream é a soma das capacidades dos seus shards.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Caso você tenha um aumento na quantidade de dados nessa ingestão, é possível aumentar ou reduzir o número de shards alocados em seu stream.

Chaves de Partição

Uma chave de partição tem a função de agrupar os dados por shards em um flow. O Kinesis separa os registros de dados pertencentes a um stream em vários shards. E usa a chave de partição associada a cada registro de dados para determinar a qual shard pertence um determinado registro de dados.

Chaves de partição são strings Unicode com um limite máximo de 256 caracteres para cada chave. Uma função hash MD5 é usada para mapear as chaves de partição para valores inteiros de 128 bits e mapear registros de dados associados a shards usando os intervalos de chaves hash dos shards. Sempre que uma aplicação coloca dados em um flow, ela deve especificar uma chave de partição.

Números Sequenciais

Cada registro possui um número sequencial exclusivo por chave de partição em seu shard. O Kinesis atribui o número de sequência depois que você grava no stream.

Vale ressaltar que os números sequenciais não podem ser usados como índices para conjuntos de dados no mesmo flow. Para uma separação lógica de dados, use chaves de partição ou crie um flow separado para cada conjunto de dados.

Biblioteca Client Kinesis

Para gerar uma aplicação de consumo de dados tolerante a falhas usando Kinesis, existe a biblioteca Client Kinesis, que garante que para cada shard haja um processador de registros executando e processando esse shard. A biblioteca também tem o papel de simplificar a leitura do flow de dados. A biblioteca Kinesis Cliente usa uma tabela do Amazon DynamoDB para armazenar uma tabela de controle por aplicação que processa dados.

E essa é mais uma ferramenta com muito mercado e bastante popular no mercado de dados. Espero que tenham visto que temos diversas opções disponíveis, mas não há uma ferramenta totalmente exclusiva, cada uma tem

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

vários pontos positivos e pontos negativos, vale uma análise para ver qual ferramenta mais adequada para o projeto que será usado.

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

2 HANDS-ON

Para começar nossa parte prática, vamos usar o Kafka em uma máquina virtual no Oracle Cloud. Esse tutorial segue com o Apache Kafka versão 3.6.1, caso queira mudar a versão ou em algum momento deixe de funcionar, pode ser por conta de mudança de versão, mas basta alterar a versão do CURL que volta a funcionar. As versões disponíveis do Kafka estão no link https://downloads.apache.org/kafka/

O primeiro passo é acessar nossa máquina virtual. Estou fazendo isso da minha máquina Windows e usarei o PowerShell. Para fazer isso, vamos usar o arquivo de conexão ssh que baixamos na criação da máquina virtual.

O meu arquivo de conexão ssh encontra-se na pasta downloads, por isso naveguei até a pasta downloads e o código abaixo nos conecta na máquina virtual via terminal. Não se esqueça de alterar o Ubuntu pelo usuário e o IP da sua máquina virtual.

```
PS C:\Users\leandro.romualdo\Downloads> ssh -i .\ssh-key-2024-01-04.key ubuntu@144.22.187.38
```

Código-fonte 1 - Conexão SSH com servidor Linux.
Fonte: Elaborado pelo autor

```
ubuntu@vm-linux-docker-kafka:~$ sudo apt install openjdk-11-jdk
```

Código-fonte - 2 Instalação do JDK 11
Fonte: Elaborado pelo autor

Vamos executar um java –version para checar se está tudo certo com a instalação e versão mais atual.

```
ubuntu@vm-linux-docker-kafka:~$ java --version
openjdk 11.0.21 2023-10-17
OpenJDK Runtime Environment (build 11.0.21+9-post-Ubuntu-0ubuntu122.04)
OpenJDK 64-Bit Server VM (build 11.0.21+9-post-Ubuntu-0ubuntu122.04, mixed mode, sharing)
```

Figura 19 - Checando versão do Java
Fonte: Elaborado pelo autor

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Vamos rodar um apt get update para atualizar os pacotes do Linux.

```
ubuntu@vm-linux-docker-kafka:~$ sudo apt-get update
```

Código-fonte 3 - Atualizando os pacotes do Linux
Fonte: Elaborado pelo autor

Feito isso, vamos criar um usuário chamado Kafka.

```
ubuntu@vm-linux-docker-kafka:~$ sudo adduser Kafka
```

Código-fonte 4 - Adicionando usuário Kafka
Fonte: Elaborado pelo autor

Agora vamos adicionar o usuário Kafka ao grupo sudo para ter privilégios de instalar o Kafka.

```
ubuntu@vm-linux-docker-kafka:~$ sudo adduser Kafka sudo
```

Código-fonte 5 - Atribuindo permissões de sudo ao usuário Kafka
Fonte: Elaborado pelo autor

Com o usuário Kafka com as permissões necessárias, vamos mudar nossa conexão para ele.

```
ubuntu@vm-linux-docker-kafka:~$ su –l Kafka
```

Código-fonte 6 - Mudando conexão para usuário Kafka
Fonte: Elaborado pelo autor

Vamos criar um diretório chamado Downloads.

```
kafka@vm-linux-docker-kafka:~$ mkdir ~/Downloads
```

Código-fonte 7 - Cria diretório Downloads
Fonte: Elaborado pelo autor

Baixaremos os arquivos de instalação do Kafka.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
kafka@vm-linux-docker-kafka:~$ curl
"https://downloads.apache.org/kafka/3.5.1/kafka_2.13-3.5.1.tgz" -o
~/Downloads/kafka.tgz
```

Código-fonte 8 - Download do Kafka
Fonte: Elaborado pelo autor

Agora vamos criar um diretório chamado Kafka, onde ficará a instalação do Kafka, e vamos acessar esse novo diretório.

```
kafka@vm-linux-docker-kafka:~/Downloads$ mkdir ~/Kafka && cd ~/kafka
```

Código-fonte 9 - Cria diretório Kafka e acessa o diretório
Fonte: Elaborado pelo autor

Vamos descompactar o arquivo usando o comando TAR.

```
kafka@vm-linux-docker-kafka:~/kafka$ tar -xvzf ~/Downloads/kafka.tgz --
strip 1
```

Código-fonte 10 - Descompacta os binários do Kafka
Fonte: Elaborado pelo autor

Ao executar o comando acima, é esperado que diversos arquivos sejam listados no terminal.

```
kafka@vm-linux-docker-kafka:~/kafka$ tar -xvzf ~/Downloads/kafka.tgz --strip 1
kafka-3.6.1-src/.asf.yaml
kafka-3.6.1-src/.github/
kafka-3.6.1-src/.github/workflows/
kafka-3.6.1-src/.github/workflows/stale.yml
kafka-3.6.1-src/.gitignore
kafka-3.6.1-src/CONTRIBUTING.md
kafka-3.6.1-src/HEADER
kafka-3.6.1-src/Jenkinsfile
kafka-3.6.1-src/LICENSE
kafka-3.6.1-src/LICENSE-binary
kafka-3.6.1-src/NOTICE
kafka-3.6.1-src/NOTICE-binary
kafka-3.6.1-src/PULL_REQUEST_TEMPLATE.md
kafka-3.6.1-src/README.md
kafka-3.6.1-src/Vagrantfile
kafka-3.6.1-src/bin/
kafka-3.6.1-src/bin/connect-distributed.sh
```

Figura 20 - Binários do Kafka
Fonte: Elaborado pelo autor

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Um comportamento padrão do Kafka é não permitir que tópicos sejam excluídos, mas esse e outros comportamentos podem ser alterados em arquivos de configuração. Para mudar isso, vamos acessar o arquivo server.properties e alterá-lo.

Execute o comando abaixo e acessará o arquivo server.properties. Aperte a tecla “i” para editar o arquivo.

```
kafka@vm-linux-docker-kafka:~/kafka$ vim ~/kafka/config/server.properties
```

Código-fonte 11 - Edita o arquivo server.properties
Fonte: Elaborado pelo autor

Aperte a tecla “i” para editar o arquivo, altere o caminho dos logs, conforme abaixo:

```
############################# Log Basics #############################

# A comma separated list of directories under which to store log files
log.dirs=/home/kafka/logs
```

Figura 21 - Alteração do diretório de logs do Kafka
Fonte: Elaborado pelo autor

Adicione uma linha, conforme abaixo:

Delete.topic.enable=true

```
############################# Group Coordinator Settings #############################

# The following configuration specifies the time, in milliseconds, that the GroupCoordi
# The rebalance will be further delayed by the value of group.initial.rebalance.delay.m
# The default value for this is 3 seconds.
# We override this to 0 here as it makes for a better out-of-the-box experience for dev
# However, in production environments the default value of 3 seconds is more suitable a
pplication startup.
group.initial.rebalance.delay.ms=0
delete.topic.enable=true

:wq
```

Figura 22 - Adiciona tópico que permite deletar tópicos
Fonte: Elaborado pelo autor

Para sair do VIM, basta apertar a tecla ESC, digitar :wq e ENTER. W vem de Write, e Q de Quit.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Agora vamos criar arquivos para o serviço do Kafka. Esses arquivos nos ajudarão a iniciar, pausar, parar o serviço do Kafka no Linux. E o Kafka usa Zookeeper para gerenciamento do cluster e outras tarefas administrativas.

Aperte a tecla “i” e adicione esse conteúdo no arquivo:

```
[Unit]
Requires=network.target remote-fs.target
After=network.target remote-fs.target

[Service]
Type=simple
User=kafka
ExecStart=/home/kafka/kafka/bin/zookeeper-server-start.sh
/home/kafka/kafka/config/zookeeper.properties
ExecStop=/home/kafka/kafka/bin/zookeeper-server-stop.sh
Restart=on-abnormal

[Install]
WantedBy=multi-user.target
```

Código-fonte 12 - Arquivo de configuração do Zookeeper
Fonte: Elaborado pelo autor

Para sair do VIM, basta apertar a tecla ESC, digitar wq e ENTER. W vem de Write, e Q de Quit.

Basicamente, nesse arquivo que criamos acima estamos definindo o funcionamento do Zookeeper, definindo os diretórios, rede e afins.

Agora vamos criar o arquivo de serviço do Kafka. Execute o comando abaixo:

```
kafka@vm-linux-docker-kafka:~/kafka$ sudo vim /etc/systemd/system/kafka.service
```

Código-fonte 13 - Cria arquivo de serviço do Kafka
Fonte: Elaborado pelo autor

Aperte a tecla “i” para editar o arquivo e adicione este conteúdo no arquivo.

```
[Unit]
Requires=zookeeper.service
After=zookeeper.service[Service]
Type=simple
User=kafka
ExecStart=/bin/sh -c '/home/kafka/kafka/bin/kafka-server-start.sh
/home/kafka/kafka/config/server.properties > /home/kafka/kafka/kafka.log 2>&1'
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
ExecStop=/home/kafka/kafka/bin/kafka-server-stop.sh
Restart=on-abnormal

[Install]
WantedBy=multi-user.target
```

Código-fonte 14 - Conteúdo do arquivo de serviço do Kafka
Fonte: Elaborado pelo autor

Com todas as configurações realizadas, vamos iniciar o serviço do Kafka.

```
kafka@vm-linux-docker-kafka:~/kafka$ sudo systemctl start kafka
```

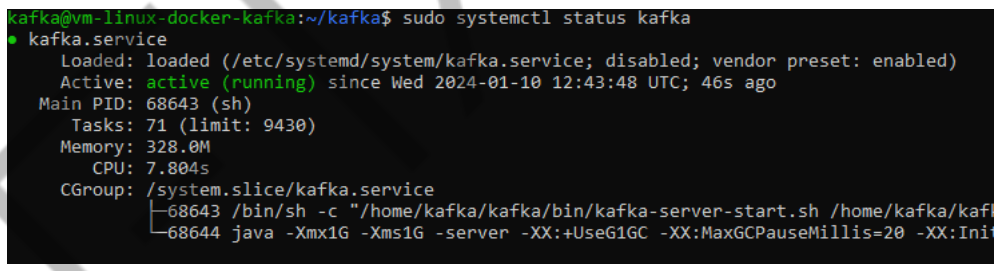
Código-fonte 15 - Iniciando o serviço do Kafka
Fonte: Elaborado pelo autor

E agora vamos checar se o serviço do Kafka está ativo.

```
kafka@vm-linux-docker-kafka:~/kafka$ sudo systemctl status kafka
```

Código-fonte 16 - Checando status do serviço do Kafka
Fonte: Elaborado pelo autor

Deve ser apresentado algo como na imagem abaixo, informando que o Kafka está ativo.



```
kafka@vm-linux-docker-kafka:~/kafka$ sudo systemctl status kafka
● kafka.service
     Loaded: loaded (/etc/systemd/system/kafka.service; disabled; vendor preset: enabled)
     Active: active (running) since Wed 2024-01-10 12:43:48 UTC; 46s ago
   Main PID: 68643 (sh)
      Tasks: 71 (limit: 9430)
     Memory: 328.0M
        CPU: 7.804s
     CGroup: /system.slice/kafka.service
             ├─68643 /bin/sh -c "/home/kafka/kafka/bin/kafka-server-start.sh /home/kafka/kaf
             └─68644 java -Xmx1G -Xms1G -server -XX:+UseG1GC -XX:MaxGCPauseMillis=20 -XX:Ini
```

Figura 23 - Status do serviço do Kafka
Fonte: Elaborado pelo autor

Agora iniciamos o Zookeeper.

```
kafka@vm-linux-docker-kafka:~/kafka$ sudo systemctl enable zookeeper
```

Código-fonte 17 - Habilitando o Zookeeper
Fonte: Elaborado pelo autor

E habilitamos o Kafka para uso.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
kafka@vm-linux-docker-kafka:~/kafka$ sudo systemctl enable Kafka
```

Código-fonte 18 - Habilitando o Kafka
Fonte: Elaborado pelo autor

Agora vamos testar nosso Kafka para ver se está funcionando.

A primeira coisa a fazer é criar um tópico, que nesse caso eu chamei de FIAP-ON com o comando abaixo:

```
kafka@vm-linux-docker-kafka:~/kafka$ ~/kafka/bin/kafka-topics.sh --create --bootstrap-server localhost:9092 --replication-factor 1 --partitions 1 --topic FIAP-ON
```

Código-fonte 19 - Cria um tópico no Kafka
Fonte: Elaborado pelo autor

Uma mensagem de que o tópico foi criado será apresentada.

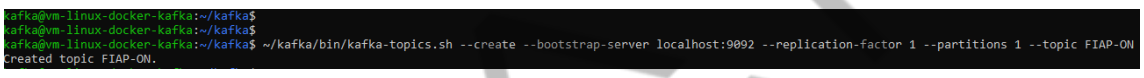


Figura 24 - Criado o tópico no Kafka
Fonte: Elaborado pelo autor

Agora com o tópico criado, vamos usar o producer para capturar um evento, nesse caso, estou capturando uma mensagem gerada por mim, e isso deve ser apresentado no consumer.

Aqui temos o script do producer. O echo vai submeter essa mensagem que escrevi entre as aspas o pipe | chama o script producer.sh passando o broker que está no localhost e porta, passamos o tópico.

```
kafka@vm-linux-docker-kafka:~/kafka$ echo "Aula de ingestão de dados Stream" | ~/kafka/bin/kafka-console-producer.sh --broker-list localhost:9092 --topic FIAP-ON > /dev/null
```

Código-fonte 20 - Adiciona um evento com texto no producer do Kafka
Fonte: Elaborado pelo autor

Para vermos o que foi passado pelo producer, chamamos o consumer nesse tópico que foi criado.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
kafka@vm-linux-docker-kafka:~/kafka$ ~/kafka/bin/kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic FIAP-ON --from-beginning
```

Código-fonte 21 - Leitura do evento com texto pelo consumer do Kafka

Fonte: Elaborado pelo autor

Na imagem abaixo tenho duas janelas do servidor abertas, a imagem da esquerda com o producer enviando as mensagens e a imagem da direita o consumer apresentando as mensagens.

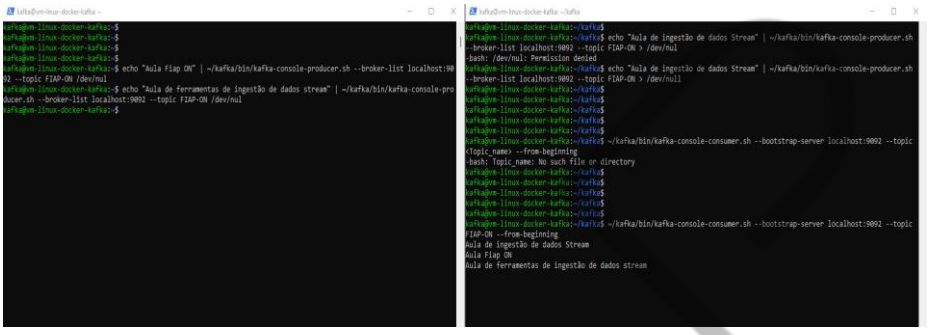


Figura 25 Producer e Consumer no Kafka em funcionamento

Fonte: Elaborado pelo autor

Esse foi nosso hands-on. Mesmo sendo uma ferramenta robusta e com muito potencial, podemos ver como foi fácil instalar e criar seus processos. Esse tutorial foi criado usando a máquina virtual no Oracle Cloud, mas pode ser replicado em qualquer máquina virtual ou física que utilize Ubuntu 20.04. Caso não use máquina virtual, desconsidere a conexão via PowerShell e use o terminal do próprio Linux. O restante do processo é o mesmo.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

REFERÊNCIAS

AMAZON Kinesis Data Stream Terminology and Concepts. **AWS Docs**, 2023. Disponível em: <https://docs.aws.amazon.com/streams/latest/dev/key-concepts.html>. Acesso em: 02 jan. 2024.

BEM-vindo ao Azure Stream Analytics. **Learn Microsoft**, 2023. Disponível em <https://learn.microsoft.com/pt-br/azure/stream-analytics/stream-analytics-introduction>. Acesso em 28 dez. 2023.

DATA Management Association International. **DMBOK**, 2023. Disponível em: <https://technicspub.com/dmbok/>. Acesso em: 03 dez. 2023.

KAFKA Academic References and Papers. **Kafka Apache**, 2023. Disponível em: <https://kafka.apache.org/books-and-papers>. Acesso em 27 dez. 2023.

KAFKA Getting Started. **Kafka documentation**, 2023. Disponível em: <https://kafka.apache.org/documentation/#gettingStarted>. Acesso em 27 dez. 2023.

KINESIS Data Streams. **AWS Docs**, 2023. Disponível em: <https://docs.aws.amazon.com/pt_br/iot/latest/developerguide/kinesis-rule-action.html>. Acesso em: 02 jan. 2024.

LEARN Flink: Hands-On Training. **Nightlies Apache**, 2023. Disponível em: <https://nightlies.apache.org/flink/flink-docs-release-1.18/docs/learn-flink/overview/>. Acesso em 26 dez.2023.

MACEY, Tobias. 97 **Things every Data Engineering Shoould Know,** 2021. Disponível em: <https://learning.oreilly.com/library/view/97-things-every/9781492062400/ch03.html>. Acesso em: 03 dez.2023.

NEEDHAM, Mark. **Building Real-Time Analytics Systems,** 2023. Disponível em: <https://learning.oreilly.com/library/view/building-real-time-analytics/9781098138783/>. Acesso em: 04 dez.2023.

PROCESSAMENTO de fluxo sem código com o Azure Stream Analytics. **Microsoft Learn,** 2023. Disponível em: <https://learn.microsoft.com/pt-br/azure/stream-analytics/no-code-stream-processing>. Acesso em 02 jan. 2024.

REIS, Joe; Housley, Matt. **Fundamentals of Data Engineering,** 2022. Disponível em: <https://learning.oreilly.com/library/view/fundamentals-of-

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

data/9781098108298/ch01.html#the_background_and_skills_of_a_data_eng/>. Acesso em: 04 dez. 2023.

Sridhar, Alla. **Fundamentals of Apache Flink,** 2019. Disponível em: <https://learning.oreilly.com/course/fundamentals-of-apache/9781789539332/>. Acesso em: 26 dez.2023.

TZOUMAS, Kostas; FRIEDMAN, Allen. **Introduction to Apache Flink**, 2016. Disponível em: <https://learning.oreilly.com/library/view/introduction-to-apache/9781491977132/preface01.html>. Acesso em: 26 dez. 2023.

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com